XAI_metrics

Maria Teresa Alba Rueda

Jul 21, 2026

API

1 xai_metrics.base package **1**
- 1.1 Submodules . . . 1
  - 1.1.1 xai_metrics.base.base_explainer module . . . 1
  - 1.1.2 xai_metrics.base.base_metric module . . . 3
  - 1.1.3 xai_metrics.base.explainer_registry module . . . 4
  - 1.1.4 xai_metrics.base.metric_registry module . . . 5
  - 1.1.5 xai_metrics.base.types module . . . 6
- 1.2 Module contents . . . 6

2 xai_metrics.config package **7**
- 2.1 Submodules . . . 7
  - 2.1.1 xai_metrics.config.config_controller module . . . 7
- 2.2 Module contents . . . 9

3 xai_metrics.explainers package **11**
- 3.1 Submodules . . . 11
  - 3.1.1 xai_metrics.explainers.autodiscover module . . . 11
  - 3.1.2 xai_metrics.explainers.breakdown module . . . 11
  - 3.1.3 xai_metrics.explainers.lime module . . . 12
  - 3.1.4 xai_metrics.explainers.maple module . . . 14
  - 3.1.5 xai_metrics.explainers.shap module . . . 16
- 3.2 Module contents . . . 17

4 xai_metrics.metrics package **19**
- 4.1 Subpackages . . . 19
  - 4.1.1 xai_metrics.metrics.complexity package . . . 19
    - 4.1.1.1 Submodules . . . 19
    - 4.1.1.2 Module contents . . . 21
  - 4.1.2 xai_metrics.metrics.faithfulness package . . . 21
    - 4.1.2.1 Submodules . . . 21
    - 4.1.2.2 Module contents . . . 30
  - 4.1.3 xai_metrics.metrics.fidelity package . . . 30
    - 4.1.3.1 Subpackages . . . 30
    - 4.1.3.2 Module contents . . . 33
  - 4.1.4 xai_metrics.metrics.robustness package . . . 33
    - 4.1.4.1 Submodules . . . 33
    - 4.1.4.2 Module contents . . . 38
  - 4.1.5 xai_metrics.metrics.sensitivity package . . . 38
    - 4.1.5.1 Submodules . . . 38
    - 4.1.5.2 Module contents . . . 40

4.2 Submodules . . . 40
4.2.1 xai_metrics.metrics.autodiscover module . . . 40
4.3 Module contents . . . 40

5 xai_metrics.reporting package **41**
5.1 Submodules . . . 41
5.1.1 xai_metrics.reporting.reporting module . . . 41
5.2 Module contents . . . 43

6 xai_metrics.runner package **45**
6.1 Submodules . . . 45
6.1.1 xai_metrics.runner.runner module . . . 45
6.2 Module contents . . . 47

Python Module Index **49**

Index **51**

CHAPTER

ONE

XAI_METRICS.BASE PACKAGE

1.1 Submodules

1.1.1 xai_metrics.base.base_explainer module

class xai_metrics.base.base_explainer.**BaseExplainer**(*context*, *params=None*)

Bases: `object`

Base class for explanation methods.

Explainer classes should inherit from this class, define their own `NAME` and implement *explain()*, which generates attribution values for a batch of inputs.

Instances are callable, so calling an explainer is equivalent to calling *explain()*. They can also expose their explanation method through *as_explain_func()*, which returns a metric-compatible function for metrics that need to recompute explanations on perturbed inputs.

Parameters

- **context** (*ExplainerContext*) – Shared context containing the background data and optional model, batch, labels and device information used by the explainer.
- **params** (*Mapping[str, Any] or None, optional*) – Explainer-specific configuration parameters. A copy of the mapping is stored in `params`. If `None`, an empty dictionary is used.

NAME: str = 'explainer'

as_explain_func()

Return the explanation method as a metric-compatible function.

The returned bound method can be passed as an `explain_func` dependency to metrics that need to generate explanations for modified or perturbed inputs.

Returns

Bound *explain()* method of the current explainer instance.

Return type

ExplainFunc

explain(*model*, *inputs*, *targets=None*, ***kwargs*)

Generate explanations for the provided inputs.

Subclasses must override this method with the logic required by the corresponding explanation method.

Parameters

- **model** (*torch.nn.Module*) – Model whose predictions are to be explained.

- **inputs** (*Any*) – Input observation or batch of observations to explain.
- **targets** (*Any or None, optional*) – Target outputs, classes or labels for which explanations are generated. Their interpretation depends on the explainer.
- ****kwargs** (*Any*) – Additional arguments required by the concrete explanation method.

Returns
Attribution values generated for the provided inputs.

Return type
numpy.ndarray

Raises
NotImplementedError – Always raised by the base implementation. Subclasses must override this method.

class xai_metrics.base.base_explainer.**ExplainerContext**(*X_background*, *y_background=None*, *model=None*, *X_batch=None*, *y_batch=None*, *device=None*)

Bases: object

Shared context used by explainer implementations.

This dataclass stores the background data required by explanation methods and, optionally, the model, current batch, target values and device used during explanation generation.

Parameters

- **X_background** (*pandas.DataFrame*) – Background or reference input data used by the explainer.
- **y_background** (*pandas.Series or None, optional*) – Labels or targets associated with X_background. This is only required by explainers that need background target values.
- **model** (*Any or None, optional*) – Model associated with the explainer context. This can be used by explainers that store the model in the context instead of receiving it only at call time.
- **X_batch** (*pandas.DataFrame or None, optional*) – Current batch of observations to be explained.
- **y_batch** (*pandas.Series or None, optional*) – Labels or targets associated with X_batch.
- **device** (*str or None, optional*) – Device used for model execution, such as "cpu" or "cuda".

X_batch: DataFrame | None = None

device: str | None = None

model: Any | None = None

y_background: Series | None = None

y_batch: Series | None = None

exception xai_metrics.base.base_explainer.**ExplainerSkipped**

Bases: Exception

Exception raised when an explainer cannot generate attributions.

This exception should be used when an explainer is not applicable to the current context, model, data, targets, or configuration.

1.1.2 xai_metrics.base.base_metric module

class xai_metrics.base.base_metric.**BaseMetric**(*context*, *params=None*)

Bases: object

Base class for all metric implementations.

Metric classes should inherit from this class, define their own NAME, and implement the *run()* method.

Parameters

- **context** (MetricContext) – Shared context containing the model, test data, labels, observations, attribution values and optional device information.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific configuration parameters. If None, an empty dictionary is used.

NAME: str = 'metric'

run()

Execute the metric computation.

Subclasses must override this method with the actual metric logic.

Returns
Metric result returned by the concrete implementation.

Return type
Any

Raises
NotImplementedError – If the subclass does not implement this method.

class xai_metrics.base.base_metric.**MetricContext**(*model*, *X_test*, *y_test*, *observations*, *attributions*, *device=None*)

Bases: object

Shared context used by metric implementations.

This dataclass stores the model, test data, labels, selected observations, attribution values, and optional extra information required by metric classes.

Parameters

- **model** (*torch.nn.Module*) – Model evaluated by the metrics.
- **X_test** (*pandas.DataFrame*) – Test input data used for metric computation.
- **y_test** (*pandas.Series*) – Test labels associated with X_test.
- **observations** (*Any*) – Identifiers of the observations explained by the attribution matrix. These values usually correspond to indexes in X_test and y_test.
- **attributions** (*numpy.ndarray*) – Attribution values for the selected observations. Each row usually corresponds to one observation and each column to one feature.
- **device** (*str or None*) – Device where the model is placed, such as "cpu" or "cuda". If None, no explicit device was configured.

device: str | None = None

exception xai_metrics.base.base_metric.**MetricSkipped**

Bases: Exception

Exception raised when a metric cannot be computed.

This exception should be used when a metric is not applicable to the current context, model, data, or attribution values.

1.1.3 xai_metrics.base.explainer_registry module

xai_metrics.base.explainer_registry.**build_explainers_from_config**(*explainers_cfg*, *context*)

Build explainer instances from a configuration list.

Each explainer configuration must contain a `name` field matching a registered explainer. It may also contain a `params` field with explainer-specific parameters.

Parameters

- **explainers_cfg** (*List[Mapping[str, Any]]*) – List of explainer configuration dictionaries. Each dictionary must contain the explainer `name` and may contain `params`.
- **context** (ExplainerContext) – Shared explainer context passed to every explainer instance.

Returns
List of instantiated explainer objects.

Return type
List[*BaseExplainer*]

Raises

- **ValueError** – If an explainer name from `explainers_cfg` is not registered.
- **KeyError** – If an explainer configuration does not contain the required `name` field.

xai_metrics.base.explainer_registry.**list_explainers**()

Return the names of the registered explainers.

Returns
Sorted list containing the names of all explainers currently registered in `EXPLAINER_REGISTRY`.

Return type
List[str]

xai_metrics.base.explainer_registry.**register_explainer**(*cls*)

Register an explainer class in the global explainer registry.

The explainer is registered using its `NAME` attribute. If the class does not define `NAME`, the class name is used instead.

Parameters
cls (*Type[BaseExplainer]*) – Explainer class to register.

Returns
The same explainer class passed as input. This allows the function to be used as a decorator.

Return type
Type[*BaseExplainer*]

Raises
ValueError – If another explainer with the same name is already registered.

Examples

```
>>> @register_explainer
... class MyExplainer(BaseExplainer):
...     NAME = "my_explainer"
...     def explain(self, model, inputs, targets=None, **kwargs):
...         return inputs
```

1.1.4 xai_metrics.base.metric_registry module

xai_metrics.base.metric_registry.**build_metrics_from_config**(*metrics_cfg*, *context*, *dependencies=None*)

Build metric instances from a configuration list.

Each metric configuration must contain a `name` field matching a registered metric. It may also contain a `params` field with metric-specific parameters. Additional dependencies are injected only if their names are accepted by the metric class constructor.

Parameters

- **metrics_cfg** (*List[Mapping[str, Any]]*) – List of metric configuration dictionaries. Each dictionary must contain the metric `name` and may contain `params`.
- **context** (MetricContext) – Shared metric evaluation context passed to every metric instance.
- **dependencies** (*Mapping[str, Any] or None, optional*) – Optional dependency objects that can be injected into metric constructors. Only dependencies whose names appear in the constructor signature are passed.

Returns
List of instantiated metric objects.

Return type
List[*BaseMetric*]

Raises

- **ValueError** – If a metric name from `metrics_cfg` is not registered.
- **KeyError** – If a metric configuration does not contain the required `name` field.

xai_metrics.base.metric_registry.**list_metrics**()

Return the names of the registered metrics.

Returns
Sorted list containing the names of all metrics currently registered in `METRIC_REGISTRY`.

Return type
List[str]

xai_metrics.base.metric_registry.**register_metric**(*cls*)

Register a metric class in the global metric registry.

The metric is registered using its `NAME` attribute. If the class does not define `NAME`, the class name is used instead.

Parameters
cls (*Type[BaseMetric]*) – Metric class to register.

Returns
The same metric class passed as input. This allows the function to be used as a decorator.

Return type
Type[*BaseMetric*]

Raises
ValueError – If another metric with the same name is already registered.

Examples

```
>>> @register_metric
... class MyMetric(BaseMetric):
...     NAME = "my_metric"
...     def run(self):
...         return 0.0
```

1.1.5 xai_metrics.base.types module

type xai_metrics.base.types.**ExplainFunc = Callable[[Module, Any, Any | None], ndarray]**

1.2 Module contents

CHAPTER

TWO

XAI_METRICS.CONFIG PACKAGE

2.1 Submodules

2.1.1 xai_metrics.config.config_controller module

class xai_metrics.config.config_controller.**ConfigController**(*config='config.yaml'*, *model_loader=None*)

Bases: `object`

Controller for loading configuration files and building contexts.

The controller can load configuration data from a mapping or from a YAML file. It supports direct context definitions and automatic discovery from dataset, model and attribution directories.

Metric contexts include models, test data, labels and precomputed attributions. Explainer contexts include models, background data and, optionally, batches to explain.

Parameters

- **config** (*Mapping[str, Any], str or pathlib.Path, default="config.yaml"*) – Configuration source. If a mapping is provided, it is copied directly. If a path is provided, the YAML file is loaded. The default value loads the `config.yaml` file located next to this module.
- **model_loader** (*Callable[[str or pathlib.Path], Any] or None, optional*) – Function used to load models from disk. The function must receive a model path and return the loaded model object. If `None`, *default_model_loader()* is used.

build_explainers_context(*ctx_cfg=None*)

Build an explainer context.

The context configuration must contain a model path and background input data. It may also contain background labels, a batch of inputs to explain, batch labels and a device. The method loads these objects, validates batch indexes when labels are available and returns both the `ExplainerContext` and its metadata.

Parameters

ctx_cfg (*Mapping[str, Any] or None, optional*) – Context configuration to use. If `None`, the `context` section of the loaded configuration is used. The configuration must contain `model_path` and `X_background_path`. It must also contain `dataset_name` and `model_name`. It may optionally contain `y_background_path`, `X_batch_path`, `X_test_path`, `y_batch_path`, `y_test_path` and `device`.

Returns

Explainer context and metadata dictionary.

Return type
Tuple[*ExplainerContext*, Dict[str, Any]]

Raises

- **ValueError** – If the `context` section is missing, required fields are missing, metadata are incomplete or batch indexes are inconsistent.
- **RuntimeError** – If a CUDA device is requested but CUDA is not available.

Warns
UserWarning – If a target file contains more than one column. In that case, only the first column is used.

`build_explainers_contexts()`

Build all explainer contexts defined by the configuration.

The method first obtains all explainer context configurations through `_iter_explainer_context_configs()` and then builds each corresponding `ExplainerContext`.

Returns
List of explainer contexts together with their metadata.

Return type
List[Tuple[*ExplainerContext*, Dict[str, Any]]]

`build_metric_context(ctx_cfg=None)`

Build a metric evaluation context.

The context configuration must contain paths to the model, input data, labels and attribution file. The method loads these objects, validates indexes, converts attribution values to a NumPy array, optionally moves the model to the configured device and returns both the `MetricContext` and its metadata.

Parameters
ctx_cfg (*Mapping[str, Any] or None, optional*) – Context configuration to use. If `None`, the `context` section of the loaded configuration is used. The configuration must contain `model_path`, `X_test_path`, `y_test_path` and `attributions_path`. It must also contain `dataset_name`, `model_name` and `xai_method_name`. It may optionally contain `device`.

Returns
Metric context and metadata dictionary.

Return type
Tuple[*MetricContext*, Dict[str, Any]]

Raises

- **ValueError** – If the `context` section is missing, required paths are missing, metadata are incomplete or indexes are inconsistent.
- **TypeError** – If the loaded model is not an instance of `torch.nn.Module`.
- **RuntimeError** – If a CUDA device is requested but CUDA is not available.

Warns
UserWarning – If `y_test` contains more than one column. In that case, only the first column is used.

`build_metric_contexts()`

Build all metric evaluation contexts defined by the configuration.

The method first obtains all metric context configurations through `_iter_metric_context_configs()` and then builds each corresponding `MetricContext`.

Returns
List of metric contexts together with their metadata.

Return type
List[Tuple[*MetricContext*, Dict[str, Any]]]

`get_explainers_config()`

Return the explainers configuration section.

Returns
List of explainer configuration dictionaries. If the section is not defined, an empty list is returned.

Return type
List

`get_metrics_config()`

Return the metrics configuration section.

Returns
List of metric configuration dictionaries. If the section is not defined, an empty list is returned.

Return type
List

`xai_metrics.config.config_controller.default_model_loader(model_path)`

Load a model from disk.

Files with `.pkl` or `.pickle` extensions are loaded with `pickle`. Files with `.joblib` or `.jl` extensions are loaded with `joblib`. Any other file extension is loaded with `torch.load()` using `weights_only=False`.

Parameters
model_path (*str or pathlib.Path*) – Path to the saved model file.

Returns
Loaded model object.

Return type
Any

2.2 Module contents

CHAPTER

THREE

XAI_METRICS.EXPLAINERS PACKAGE

3.1 Submodules

3.1.1 xai_metrics.explainers.autodiscover module

xai_metrics.explainers.autodiscover.**autodiscover_explainers**(*package*)

Import all modules inside an explainer package.

The function walks recursively through the package and imports every module it finds. This triggers module-level registration decorators, such as `@register_explainer`, making the discovered explainers available through the explainer registry.

Parameters

package (*ModuleType*) – Package module to inspect. It must expose `__name__` and `__path__`.

3.1.2 xai_metrics.explainers.breakdown module

class xai_metrics.explainers.breakdown.**BreakDownExplainer**(*context*, *params=None*)

Bases: *BaseExplainer*

DALEX Break Down explainer.

This explainer computes local feature attributions using DALEX `predict_parts` with `type="break_down"`. For each observation, the DALEX contributions are converted into a fixed-size attribution vector whose order follows the columns of `context.X_background`.

The explainer supports classification and regression models through a configurable prediction function. For classification, the selected output column of `predict_proba` is used.

The explainer is registered under the name `"BreakDown"`.

Parameters

- **context** (ExplainerContext) – Shared explainer context. It must contain `X_background` and may contain `y_background`. The background data columns define the order of the returned attribution vectors.
- **params** (*Mapping[str, Any] or None, optional*) – Explainer-specific parameters. Supported keys are:
  - `mode` : str, optional Model task type. If `"classification"`, the explainer uses `predict_proba`. Otherwise, it uses `predict`. The default value is `"classification"`.
  - `output_index` : int, optional Output column selected from `predict_proba` for classification models. The default value is `1`.

- label : str, optional Label passed to the DALEX explainer. If not provided, the model class name is used.
- verbose : bool, optional Whether DALEX should print additional information. The default value is False.
- precalculate : bool, optional Whether DALEX should precalculate model diagnostics. The default value is True.
- order : Any, optional Feature order passed to predict_parts.
- n_samples : int or None, optional Number of samples passed to predict_parts through N.
- keep_distributions : bool, optional Whether DALEX should keep contribution distributions. The default value is False.
- random_state : int, optional Random state passed to predict_parts. The default value is 42.
- abs : bool, optional Whether to return absolute attribution values. The default value is False.

If None, an empty dictionary is used.

NAME: str = 'BreakDown'

explain(*model*, *inputs*, *targets=None*, ***kwargs*)

Generate Break Down attributions for the provided inputs.

The method converts the input data to a dataframe, obtains a cached DALEX explainer for the model and computes one break_down explanation per observation. The DALEX contributions are converted into attribution vectors ordered according to context.X_background.

Parameters

- **model** (*Any*) – Model whose predictions are to be explained.
- **inputs** (*Any*) – Input observation or batch of observations to explain.
- **targets** (*Any or None, optional*) – Unused by this explainer. The target output is controlled through the prediction function and, in classification mode, through output_index.
- ****kwargs** (*Any*) – Additional keyword arguments. They are accepted for compatibility with the common explainer interface.

Returns

Attribution matrix with one row per explained observation and one column per feature. If abs=True, absolute attribution values are returned.

Return type

numpy.ndarray

3.1.3 xai_metrics.explainers.lime module

class xai_metrics.explainers.lime.**LIMEExplainer**(*context*, *params=None*)

Bases: *BaseExplainer*

LIME tabular explainer.

This explainer computes local feature attributions using `lime.lime_tabular.LimeTabularExplainer`. For each observation, LIME builds a local surrogate model around the input and returns feature weights for the selected target label.

The returned attributions are arranged as a fixed-size matrix whose columns follow the order of `context.X_background`. The explainer supports classification and regression models through `predict_proba` or `predict`, respectively.

The explainer is registered under the name `"LIME"`.

Parameters

- **context** (*ExplainerContext*) – Shared explainer context. It must contain `X_background` and may contain `y_background`. The background data are used to initialise the LIME tabular explainer, and their columns define the order of the returned attribution vectors.
- **params** (*Mapping[str, Any] or None, optional*) – Explainer-specific parameters. Supported keys are:
  - `mode` : str, optional LIME mode. Usually `"classification"` or `"regression"`. The default value is `"classification"`.
  - `categorical_features` : list or None, optional Indices of categorical features passed to LIME.
  - `categorical_names` : dict or None, optional Mapping from categorical feature indices to category names.
  - `kernel_width` : float or None, optional Width of the LIME kernel.
  - `kernel` : callable or None, optional Custom kernel function used by LIME.
  - `verbose` : bool, optional Whether LIME should print additional information. The default value is `False`.
  - `class_names` : list or None, optional Class names used by LIME in classification mode.
  - `feature_selection` : str, optional Feature selection strategy used by LIME. The default value is `"auto"`.
  - `discretize_continuous` : bool, optional Whether continuous features should be discretised. The default value is `True`.
  - `discretizer` : str, optional Discretisation strategy used by LIME. The default value is `"quartile"`.
  - `sample_around_instance` : bool, optional Whether perturbations should be sampled around the explained instance. The default value is `False`.
  - `random_state` : int, optional Random state used by LIME. The default value is `42`.
  - `labels` : tuple, list or None, optional Labels to explain. If not provided, the target labels passed to *explain()* are used. If no targets are provided, label `1` is used.
  - `top_labels` : int or None, optional Number of top labels to explain. When provided, the first label returned by LIME is used to build the attribution vector.
  - `num_features` : int, optional Maximum number of features included in each local explanation. The default value is the number of input features.
  - `num_samples` : int, optional Number of perturbed samples generated by LIME. The default value is `500`.
  - `distance_metric` : str, optional Distance metric used by LIME. The default value is `"euclidean"`.

- `model_regressor` : Any or None, optional Regressor used by LIME to fit the local surrogate model.

If `None`, an empty dictionary is used.

NAME: str = 'LIME'

explain(*model*, *inputs*, *targets=None*, ***kwargs*)

Generate LIME attributions for the provided inputs.

The method converts the inputs to a NumPy array, resolves the prediction function and computes one LIME explanation per observation. The selected LIME weights are converted into attribution vectors ordered according to `context.X_background`.

If `labels` and `top_labels` are not configured, the method uses the provided `targets` as labels. If no targets are provided, label 1 is explained by default.

Parameters

- **model** (*Any*) – Model whose predictions are to be explained.
- **inputs** (*Any*) – Input observation or batch of observations to explain.
- **targets** (*Any or None, optional*) – Target labels to explain when `labels` and `top_labels` are not provided in `params`.
- ****kwargs** (*Any*) – Additional keyword arguments. They are accepted for compatibility with the common explainer interface.

Returns
Attribution matrix with one row per explained observation and one column per feature.

Return type
numpy.ndarray

3.1.4 xai_metrics.explainers.maple module

class xai_metrics.explainers.maple.**MAPLEExplainer**(*context*, *params=None*)

Bases: *BaseExplainer*

MAPLE explainer.

This explainer computes local feature attributions using a MAPLE-style surrogate model. Background data are split into training and validation subsets. The training subset is used to fit the tree ensemble and local linear models, while the validation subset is used to select the retained feature subset.

For each observation, the explainer obtains the model response on the background data, builds or reuses a cached internal MAPLE model, and returns the coefficients of a local weighted Ridge model as attributions.

The explainer is based on MAPLE by Plumb, Molitor and Talwalkar (2018), "Model Agnostic Supervised Local Explanations", and is registered under the name `"MAPLE"`.

Parameters

- **context** (ExplainerContext) – Shared explainer context. It must contain `X_background`. The background data columns define the order of the returned attribution vectors.
- **params** (*Mapping[str, Any] or None, optional*) – Explainer-specific parameters. Supported keys are:
  - `mode` : str, optional Model task type. If `"classification"`, the explainer uses `predict_proba`. Otherwise, it uses `predict`. The default value is `"classification"`.

- `output_index` : int, optional Output column selected from two-dimensional model predictions. The default value is `1`.
- `validation_size` : float, optional Fraction of background observations used for validation. The default value is `0.2`.
- `data_type` : str, optional Type of background data. Supported values are `"tabular"` and `"time_series"`. For time series, the validation set is taken from the end of the background data. The default value is `"tabular"`.
- `fe_type` : str, optional Tree ensemble used by MAPLE. Supported values are `"rf"` and `"gbrt"`. The default value is `"rf"`.
- `n_estimators` : int, optional Number of trees in the ensemble. The default value is `200`.
- `max_features` : float, int, str or None, optional Maximum number of features considered by the ensemble. The default value is `0.5`.
- `min_samples_leaf` : int, optional Minimum number of samples required in each leaf. The default value is `10`.
- `regularization` : float, optional Ridge regularisation strength used by the local linear models. The default value is `0.001`.
- `random_state` : int or None, optional Random state used by the ensemble and background split. The default value is `42`.
- `abs` : bool, optional Whether to return absolute attribution values. The default value is `False`.

If `None`, an empty dictionary is used.

Raises

`ValueError` – If fewer than three background observations are available.

`NAME: str = 'MAPLE'`

`explain`(*model*, *inputs*, *targets=None*, ***kwargs*)

Generate MAPLE attributions for the provided inputs.

The method converts the inputs to a NumPy array, obtains a cached internal MAPLE model and computes one local coefficient vector per observation. These coefficient vectors are returned as feature attributions.

Parameters

- **`model`** (*Any*) – Model whose predictions are to be explained.
- **`inputs`** (*Any*) – Input observation or batch of observations to explain.
- **`targets`** (*Any or None, optional*) – Unused by this explainer. The explained output is controlled by the model prediction function and, for two-dimensional outputs, by `output_index`.
- **`**kwargs`** (*Any*) – Additional keyword arguments. They are accepted for compatibility with the common explainer interface.

Returns

Attribution matrix with one row per explained observation and one column per feature. If `abs=True`, absolute attribution values are returned.

Return type

numpy.ndarray

3.1.5 xai_metrics.explainers.shap module

class xai_metrics.explainers.shap.**SHAPExplainer**(*context*, *params=None*)

Bases: *BaseExplainer*

SHAP explainer.

This explainer computes feature attributions using `shap.Explainer`. Background data from `context.X_background` are used to initialise the SHAP explainer. For each batch of inputs, the SHAP values are converted into an attribution matrix whose columns follow the order of the background data.

For multi-output explanations, the output to explain can be selected through `output_index`, through the provided `targets` or through the context batch labels. If none of these are available, `default_output` is used.

The explainer is registered under the name `"SHAP"`.

Parameters

- **context** (*ExplainerContext*) – Shared explainer context. It must contain `X_background` and may contain `y_batch`. The background data are used to initialise the SHAP explainer, and their columns define the order of the returned attribution vectors.
- **params** (*Mapping[str, Any] or None, optional*) – Explainer-specific parameters. Supported keys are:
  - `mode` : str, optional Model task type. If `"classification"`, the explainer uses `predict_proba`. Otherwise, it uses `predict`. The default value is `"classification"`.
  - `max_background_samples` : int or None, optional Maximum number of background observations used by SHAP. If provided, the background data are subsampled with `shap.sample()`.
  - `random_state` : int, optional Random state used for background sampling and SHAP explainer initialisation. The default value is `42`.
  - `algorithm` : str, optional SHAP algorithm passed to `shap.Explainer`. The default value is `"auto"`.
  - `output_names` : list or None, optional Output names passed to `shap.Explainer`.
  - `output_index` : int or None, optional Output index used when SHAP returns multi-output values. If provided, it takes precedence over `targets` and `context.y_batch`.
  - `default_output` : int, optional Output index used when SHAP returns multi-output values and no explicit output index or targets are available. The default value is `1`.
  - `max_evals` : int or None, optional Maximum number of evaluations passed to SHAP. If `None`, `"auto"` is used.
  - `batch_size` : int or None, optional Batch size passed to SHAP. If `None`, `"auto"` is used.
  - `abs` : bool, optional Whether to return absolute SHAP values. The default value is `False`.

  If `None`, an empty dictionary is used.

NAME: str = 'SHAP'

explain(*model*, *inputs*, *targets=None*, ***kwargs*)

Generate SHAP attributions for the provided inputs.

The method converts the inputs to a NumPy array, obtains a cached SHAP explainer and computes SHAP values for the batch. Multi-output values are reduced to a single output according to `output_index`, `targets`, `context.y_batch` or `default_output`.

Parameters

- **model** (*Any*) – Model whose predictions are to be explained.
- **inputs** (*Any*) – Input observation or batch of observations to explain.
- **targets** (*Any or None, optional*) – Target labels used to select the output dimension when SHAP returns multi-output values.
- ****kwargs** (*Any*) – Additional keyword arguments. They are accepted for compatibility with the common explainer interface.

Returns

Attribution matrix with one row per explained observation and one column per feature. If `abs=True`, absolute SHAP values are returned.

Return type

numpy.ndarray

3.2 Module contents

CHAPTER

FOUR

XAI_METRICS.METRICS PACKAGE

4.1 Subpackages

4.1.1 xai_metrics.metrics.complexity package

4.1.1.1 Submodules

xai_metrics.metrics.complexity.complexity_metric module

class xai_metrics.metrics.complexity.complexity_metric.**Complexity**(*context*, *params=None*)

Bases: *BaseMetric*

Quantus Complexity metric.

This metric measures explanation complexity using the entropy of the attribution distribution. Quantus takes the absolute attribution values and expresses each feature contribution as a fraction of the total attribution magnitude.

Higher scores indicate that importance is distributed across more features and therefore corresponds to more complex explanations. Lower scores indicate that importance is concentrated on fewer features.

For an explanation with `n` features, the maximum entropy is approximately `log(n)` when importance is distributed uniformly. The scores returned by this wrapper are not divided by this maximum value.

The metric is based on the Complexity metric proposed by Bhatt et al. (2020) and implemented in Quantus.

Parameters

- **context** (MetricContext) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
  - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.

  If `None`, an empty dictionary is used.

Notes

The wrapper uses the default normalisation function provided by Quantus. Quantus also applies the absolute-value operation to the attributions because its `abs` parameter is not exposed by this wrapper and defaults to `True`.

NAME: str = 'Complexity'

run()

Compute the Complexity metric.

The method passes the selected input data, labels and attribution values to `quantus.Complexity`. Quantus flattens each explanation, takes its absolute values and computes the entropy of the fractional feature contributions.

If all attribution values are negative, this wrapper converts them to absolute values before calling Quantus. The model is set to training mode following the current wrapper implementation, although the metric itself depends only on the attribution values.

Returns

Complexity score for each evaluated observation. Lower values indicate that importance is concentrated on fewer features, while higher values indicate that it is more widely distributed. Scores are not normalised by the theoretical maximum `log(n)`.

Return type

List[float]

xai_metrics.metrics.complexity.sparseness module

class xai_metrics.metrics.complexity.sparseness.**Sparseness**(*context*, *params=None*)

Bases: *BaseMetric*

Quantus Sparseness metric.

This metric measures how concentrated the attribution magnitude is across the input features using the Gini index. Quantus applies the absolute-value operation to the attributions before computing the score.

Higher scores indicate that importance is concentrated on a smaller subset of features and therefore correspond to sparser explanations. Lower scores indicate a more uniform distribution of attribution magnitude.

The metric is based on the Sparseness metric proposed by Chalasani et al. (2020) and implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
  - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.

  If `None`, an empty dictionary is used.

Notes

The wrapper uses the default normalisation function provided by Quantus. Quantus applies the absolute-value operation to the attributions because its `abs` parameter is not exposed by this wrapper and defaults to `True`.

NAME: str = 'Sparseness'

run()

Compute the Sparseness metric.

The method passes the selected input data, labels and attribution values to `quantus.Sparseness`. Quantus flattens each explanation, takes the absolute attribution values, sorts them and computes their Gini index.

If all attribution values are negative, this wrapper converts them to absolute values before calling Quantus. The model is set to training mode following the current wrapper implementation, although the metric itself depends only on the attribution values.

Returns
Sparseness score for each evaluated observation. Higher values indicate that attribution magnitude is concentrated on fewer features, while lower values indicate a more uniform distribution.

Return type
List[float]

4.1.1.2 Module contents

4.1.2 xai_metrics.metrics.faithfulness package

4.1.2.1 Submodules

xai_metrics.metrics.faithfulness.consistency module

class xai_metrics.metrics.faithfulness.consistency.**Consistency**(*context*, *params=None*)

Bases: *BaseMetric*

Quantus Consistency metric.

This metric evaluates whether observations with equivalent explanations exhibit consistent model behaviour. Continuous attribution vectors are discretised by Quantus and observations sharing the same discrete representation are grouped together. For each observation, the metric computes the proportion of other observations in its group that receive the same predicted class.

Higher scores indicate stronger agreement between similar explanations and model predictions.

The metric is based on the Consistency metric proposed by Dasgupta et al. (2022) and implemented in Quantus.

Parameters

- **context** (MetricContext) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
  - `abs` : bool, optional Whether to use absolute attribution values before computing the metric. If all attribution values are negative and this parameter is `True`, their absolute values are used. If it is `False`, the metric is skipped. The default value is `True`.
  - `normalise` : bool, optional Whether to normalise the attribution values before discretising the explanations. The default value is `True`.

  If `None`, an empty dictionary is used.

Notes

The wrapper uses the default discretisation and normalisation functions provided by Quantus. The default discretisation function is `top_n_sign`.

NAME: str = 'Consistency'

run()

Compute the Consistency metric.

The method selects the observations defined in the metric context and passes their input data, labels and attribution values to `quantus.Consistency`. Quantus discretises the explanations, predicts the class of each observation and compares predictions among observations with equivalent discrete explanations.

If all attribution values are negative, their treatment depends on the `abs` parameter. Their absolute values are used when `abs=True`; otherwise, the metric is skipped.

The model is set to evaluation mode before the metric is computed.

Returns
Consistency score for each evaluated observation. Higher values indicate that observations with the same discretised explanation more frequently receive the same predicted class.

Return type
List[float]

Raises
MetricSkipped – If all attribution values are negative and `abs` is `False`.

xai_metrics.metrics.faithfulness.faithfulness module

class xai_metrics.metrics.faithfulness.faithfulness.**Faithfulness**(*context*, *params=None*)

Bases: *BaseMetric*

AIX360 Faithfulness metric.

This metric evaluates whether feature attribution values reflect the influence of the corresponding features on the model prediction. For each observation, every feature is individually replaced with a baseline value and the probability assigned to the original predicted class is recorded. The score is the negative Pearson correlation between the attribution values and these probabilities.

Higher scores indicate stronger agreement between feature importance and the effect of replacing features with their baseline values.

The wrapped AIX360 implementation requires a classification model exposing a `predict_proba` method.

The metric is based on the faithfulness criterion proposed by Alvarez-Melis and Jaakkola (2018) and implemented in AIX360.

Parameters

- **context** (MetricContext) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values. The model must implement `predict_proba`.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
  - `base_values` : array-like or None, optional Explicit baseline values used to replace feature values. If provided, this takes priority over `base_func` and `base_strategy`.
  - `base_strategy` : str, optional Strategy used to compute baseline values from `X_test` when `base_values` and `base_func` are not provided. Supported values are `"mean"`, `"median"` and `"zero"`. The default value is `"mean"`.

  If `None`, an empty dictionary is used.

NAME: str = 'Faithfulness'

run()

Compute the Faithfulness metric.

The method resolves a common baseline vector from the complete test dataset and evaluates each selected observation independently using `aix360.metrics.faithfulness_metric()`.

For each observation, AIX360 determines the predicted class, replaces each feature individually with its baseline value and records the resulting probability for that class. The returned score is the negative Pearson correlation between these probabilities and the attribution values.

Returns
Faithfulness score for each evaluated observation. Higher values indicate that features with larger attribution values produce greater decreases in the predicted class probability when replaced.

Return type
List[float]

Raises

- **ValueError** – If `base_strategy` is not `"mean"`, `"median"` or `"zero"`.
- **AttributeError** – If the model does not implement the `predict_proba` method required by AIX360.

Notes

The score may be `nan` when the attribution values or the probabilities obtained after feature replacement have zero variance.

xai_metrics.metrics.faithfulness.faithfulness_estimate module

class xai_metrics.metrics.faithfulness.faithfulness_estimate.**FaithfulnessEstimate**(*context*, *params=None*)

Bases: *BaseMetric*

Quantus Faithfulness Estimate metric.

This metric evaluates whether features with higher attribution values produce larger changes in the model output when they are perturbed. Features are perturbed in groups ordered by attribution importance, and the metric compares the sum of their attributions with the corresponding drop in the target output.

This wrapper uses a safe Pearson correlation function that returns `0.0` when either of the compared vectors has zero variance, avoiding undefined correlation values.

Higher scores indicate stronger agreement between attribution importance and changes in the model output.

The metric is based on the Faithfulness Estimate proposed by Alvarez-Melis and Jaakkola (2018) and implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
  - `features_in_step` : int, optional Number of features perturbed at each step. The default value is 1.

- `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
- `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.
- `perturb_baseline` : str, optional Baseline value used when perturbing features. Supported values depend on the Quantus perturbation function. Common values are `"black"`, `"white"`, `"mean"`, `"random"` and `"uniform"`. The default value is `"black"`.

If `None`, an empty dictionary is used.

Notes

This wrapper uses its internal `_safe_pearson()` method as the similarity function. The default normalisation and perturbation functions provided by Quantus are used.

`NAME: str = 'FaithfulnessEstimate'`

`run``()`

Compute the Faithfulness Estimate metric.

The method selects the observations defined in the metric context and passes their input data, labels and attribution values to `quantus.FaithfulnessEstimate`. Features are perturbed in groups, and the internal safe Pearson correlation function compares attribution sums with the resulting target-output drops.

If all attribution values are negative, their treatment depends on the `abs` parameter. Their absolute values are used when `abs=True`; otherwise, the metric is skipped.

The model is set to evaluation mode before the metric is computed.

Returns
Faithfulness Estimate score for each evaluated observation. Higher values indicate stronger agreement between attribution importance and model output changes after perturbation.

Return type
List[float]

Raises
`MetricSkipped` – If all attribution values are negative and `abs` is `False`.

xai_metrics.metrics.faithfulness.monotonicity module

`class` xai_metrics.metrics.faithfulness.monotonicity.**`Monotonicity`**(*context*, *params=None*)

Bases: *`BaseMetric`*

Quantus Monotonicity metric.

This metric evaluates whether the target model output increases monotonically as features are progressively introduced from a baseline input. Features are processed in increasing order of attribution value and may be introduced individually or in groups.

For each observation, the metric returns `True` when the sequence of target outputs is monotonically non-decreasing and `False` otherwise.

The metric is based on the Monotonicity metric proposed by Arya et al. (2019) and the monotonic attribute functions described by Luss et al. (2019), as implemented in Quantus.

Parameters

- **`context`** (`MetricContext`) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.

- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
  - `features_in_step` : int, optional Number of features added at each perturbation step. The default value is `1`.
  - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `True`.
  - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.
  - `perturb_baseline` : str, optional Baseline value used to initialise the perturbed input. Supported values depend on the Quantus perturbation function. Common values are `"black"`, `"white"`, `"mean"`, `"random"` and `"uniform"`. The default value is `"black"`.

  If `None`, an empty dictionary is used.

Notes

The wrapper uses the default normalisation and perturbation functions provided by Quantus.

NAME: str = 'Monotonicity'

run()

Compute the Monotonicity metric.

The method passes the selected input data, target labels and attribution values to `quantus.Monotonicity`. For each observation, Quantus starts from a baseline input, processes features in increasing order of attribution value and evaluates the target model output after each group is introduced.

If all attribution values are negative, their absolute values are used when `abs=True`; otherwise, the metric is skipped. The model is set to evaluation mode before the computation.

Returns
Monotonicity result for each evaluated observation. `True` indicates that the target output is monotonically non-decreasing across the feature-introduction steps.

Return type
List[bool]

Raises
MetricSkipped – If all attribution values are negative and `abs` is `False`.

xai_metrics.metrics.faithfulness.monotonicity_correlation module

class xai_metrics.metrics.faithfulness.monotonicity_correlation.**MonotonicityCorrelation**(*context*, *params=None*)

Bases: *BaseMetric*

Quantus Monotonicity Correlation metric.

This metric evaluates whether feature attribution values are monotonically related to the uncertainty caused by perturbing the corresponding features. Features are grouped in increasing order of attribution, and each group is perturbed repeatedly to estimate its effect on the target model output. The score is the Spearman correlation between the attribution sums and the estimated output variations.

This wrapper uses a safe Spearman correlation that returns `0.0` when either input vector has zero variance.

Higher scores indicate a stronger positive relationship between feature importance and the uncertainty caused by perturbing those features.

The metric is based on the Monotonicity Correlation metric proposed by Nguyen and Rodríguez Martínez (2020) and implemented in Quantus.

Parameters

- **context** (MetricContext) – Shared metric evaluation context. It must contain the model, X_test, y_test, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
  - eps : float, optional Threshold used when computing the inverse prediction factor. The default value is 1e-5.
  - nr_samples : int, optional Number of perturbation samples generated for each feature group. The default value is 100.
  - features_in_step : int, optional Number of features perturbed at each step. The default value is 1.
  - abs : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is True.
  - normalise : bool, optional Whether to normalise the attribution values before computing the metric. The default value is True.
  - perturb_baseline : str, optional Baseline value used when perturbing features. Supported values depend on the Quantus perturbation function. Common values are "black", "white", "mean", "random" and "uniform". The default value is "uniform".

  If None, an empty dictionary is used.

Notes

The wrapper uses _safe_spearman() as the similarity function and the default normalisation and perturbation functions provided by Quantus.

NAME: str = 'MonotonicityCorrelation'

run()

Compute the Monotonicity Correlation metric.

The method passes the selected inputs, labels and attribution values to quantus.MonotonicityCorrelation. Quantus perturbs groups of features repeatedly, estimates their relative effect on the target output and compares those estimates with the corresponding attribution sums using _safe_spearman().

If all attribution values are negative, their absolute values are used when abs=True; otherwise, the metric is skipped. The model is set to evaluation mode before the computation.

Returns
Monotonicity Correlation score for each evaluated observation. Higher values indicate a stronger positive relationship between attribution importance and output variation.

Return type
List[float]

Raises
MetricSkipped – If all attribution values are negative and abs is False.

xai_metrics.metrics.faithfulness.monotonicity_metric module

class xai_metrics.metrics.faithfulness.monotonicity_metric.**MonotonicityMetric**(*context*, *params=None*)

Bases: *BaseMetric*

AIX360 Monotonicity metric.

This metric evaluates whether the probability assigned to the originally predicted class increases monotonically as features are progressively restored from a baseline input.

For each observation, features are restored in increasing order of their attribution values. The metric returns `True` when the resulting sequence of predicted-class probabilities is monotonically non-decreasing.

The wrapped AIX360 implementation requires a classification model exposing a `predict_proba` method.

The metric is based on the monotonicity criterion described by Luss et al. (2019) and implemented in AIX360.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values. The model must implement `predict_proba`.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
  - `base_values` : array-like or None, optional Explicit baseline values used as the initial feature values. If provided, this takes priority over `base_func` and `base_strategy`.
  - `base_strategy` : str, optional Strategy used to compute baseline values from `X_test` when `base_values` and `base_func` are not provided. Supported values are `"mean"`, `"median"` and `"zero"`. The default value is `"mean"`.

  If `None`, an empty dictionary is used.

NAME: str = 'MonotonicityMetric'

run()

Compute the Monotonicity metric.

The method resolves a common baseline vector from the complete test dataset and evaluates each selected observation independently using `aix360.metrics.monotonicity_metric()`.

For each observation, AIX360 determines the class predicted for the original input. Starting from the baseline, it restores features cumulatively in increasing order of attribution value and evaluates the probability assigned to that class after each restoration.

Returns
Monotonicity result for each evaluated observation. `True` indicates that the predicted-class probability never decreases as features are progressively restored.

Return type
List[bool]

Raises

- **ValueError** – If `base_strategy` is not `"mean"`, `"median"` or `"zero"`.
- **AttributeError** – If the model does not implement the `predict_proba` method required by AIX360.

xai_metrics.metrics.faithfulness.sensitivity_n module

class xai_metrics.metrics.faithfulness.sensitivity_n.**SensitivityN**(*context*, *params=None*)

Bases: *BaseMetric*

Quantus Sensitivity-N metric.

This metric evaluates the agreement between feature attribution values and the variation in the target model output caused by perturbing the corresponding features.

Quantus orders features by decreasing attribution value and progressively perturbs them in groups. At each step, it compares the target-output change with the attribution sum of the processed feature group using Pearson correlation across the evaluated observations.

The number of perturbation steps is limited by `n_max_percentage`. Higher correlation values indicate stronger agreement between the explanation and the model behaviour.

The metric is based on the Sensitivity-N test proposed by Ancona et al. (2018) and implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
  - `n_max_percentage` : float, optional Maximum percentage of features to evaluate. The default value is `0.8`.
  - `features_in_step` : int, optional Number of features perturbed at each step. The default value is `1`.
  - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
  - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.
  - `perturb_baseline` : str, optional Baseline value used when perturbing features. Supported values depend on the Quantus perturbation function. Common values are `"black"`, `"white"`, `"mean"`, `"random"` and `"uniform"`. The default value is `"black"`.

  If `None`, an empty dictionary is used.

Notes

The wrapper uses the default Pearson correlation, normalisation and baseline-replacement functions provided by Quantus. Quantus aggregates the step-wise correlations by default.

NAME: str = 'SensitivityN'

run()

Compute the Sensitivity-N metric.

The method passes the selected input data, target labels and attribution values to `quantus.SensitivityN`. Quantus progressively perturbs feature groups ordered by decreasing attribution value and computes the Pearson correlation between their attribution sums and the resulting target-output changes.

Only the perturbation steps covered by `n_max_percentage` are evaluated. If all attribution values are negative, their absolute values are used when `abs=True`; otherwise, the metric is skipped. The model is set to evaluation mode before the computation.

Returns
Sensitivity-N result returned by Quantus. Higher values indicate stronger agreement between attribution values and target-output changes. With the default Quantus configuration, the step-wise correlations are aggregated.

Return type
List[float]

Raises
MetricSkipped – If all attribution values are negative and `abs` is `False`.

xai_metrics.metrics.faithfulness.sufficiency module

class xai_metrics.metrics.faithfulness.sufficiency.**Sufficiency**(*context*, *params=None*)

Bases: *BaseMetric*

Quantus Sufficiency metric.

This metric evaluates whether observations with similar explanations receive the same model prediction. Quantus computes the pairwise distances between attribution vectors and considers two explanations similar when their distance is less than or equal to `threshold`.

For each observation, the score is the proportion of other observations with similar explanations that receive the same predicted class. A score of `0.0` is returned when no other explanation satisfies the distance threshold.

Higher scores indicate stronger agreement between similar explanations and model predictions. Since explanations are compared within the evaluated batch, the results depend on the selected observations.

The metric is based on the Sufficiency test proposed by Dasgupta et al. (2022) and implemented in Quantus.

Parameters

- **context** (MetricContext) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
  - `threshold` : float, optional Maximum distance between two attribution vectors for their explanations to be considered similar. The default value is `0.6`.
  - `distance_func` : str, optional Distance function used to compare attribution vectors. The value is passed to Quantus and is typically a valid SciPy distance name, such as `"seuclidean"`. The default value is `"seuclidean"`.
  - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `True`.
  - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.

  If `None`, an empty dictionary is used.

Notes

The wrapper uses the default normalisation function provided by Quantus. Scores depend on the observations evaluated in the same call, since these observations define the explanation neighbourhoods.

NAME: str = 'Sufficiency'

run()

Compute the Sufficiency metric.

The method passes the selected input data, labels and attribution values to `quantus.Sufficiency`. Quantus compares the attribution vectors, identifies observations with similar explanations and computes the proportion that receive the same predicted class. Self-comparisons are excluded.

If all attribution values are negative, their absolute values are used when `abs=True`; otherwise, the metric is skipped. The model is set to evaluation mode before the computation.

Returns
Sufficiency score for each evaluated observation. Scores range from `0.0` to `1.0`. Higher values indicate that observations with similar explanations more frequently receive the same predicted class.

Return type
List[float]

Raises
MetricSkipped – If all attribution values are negative and `abs` is `False`.

4.1.2.2 Module contents

4.1.3 xai_metrics.metrics.fidelity package

4.1.3.1 Subpackages

xai_metrics.metrics.fidelity.completeness package

Submodules

xai_metrics.metrics.fidelity.completeness.completeness_metric module

class xai_metrics.metrics.fidelity.completeness.completeness_metric.**Completeness**(*context*, *params=None*)

Bases: *BaseMetric*

Quantus Completeness metric.

This metric evaluates whether the sum of the attribution values matches the difference between the model output for the original input and the model output for a baseline input. This property is also known as Summation to Delta or Conservation.

The metric returns one boolean value per observation. A value of `True` indicates that the attribution sum satisfies the completeness condition for the selected model output.

Higher scores are better when the boolean results are aggregated, since a larger proportion of `True` values indicates stronger agreement with the completeness axiom.

The metric is based on the Completeness property proposed by Sundararajan et al. (2017), the Summation to Delta property proposed by Shrikumar et al. (2017), and the Conservation property discussed by Montavon et al. (2018), as implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:

- `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
- `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.
- `perturb_baseline` : str, optional Baseline value used to create the reference input. Supported values depend on the Quantus perturbation function. Common values are `"black"`, `"white"`, `"mean"`, `"random"` and `"uniform"`. The default value is `"black"`.

If `None`, an empty dictionary is used.

Notes

The wrapper uses the default functions provided by Quantus. The default perturbation function replaces all features by the configured baseline, and the default output transformation is the identity function.

Quantus computes this metric using logits by default, not softmax probabilities.

NAME: str = 'Completeness'

run()

Compute the Completeness metric.

The method selects the observations defined in the metric context and passes their input data, labels and attribution values to `quantus.Completeness`. Quantus replaces the input features by a baseline, computes the difference between the model output at the original input and at the baseline input, and compares this difference with the sum of the attribution values.

If all attribution values are negative, their treatment depends on the `abs` parameter. Their absolute values are used when `abs=True`; otherwise, the metric is skipped.

The model is set to evaluation mode before the metric is computed.

Returns
Completeness result for each evaluated observation. `True` indicates that the sum of the attribution values matches the output difference between the original input and the baseline input.

Return type
List[bool]

Raises
MetricSkipped – If all attribution values are negative and `abs` is `False`.

Module contents

xai_metrics.metrics.fidelity.soundness package

Submodules

xai_metrics.metrics.fidelity.soundness.non_sensitivity module

class xai_metrics.metrics.fidelity.soundness.non_sensitivity.**NonSensitivity**(*context*, *params=None*)

Bases: *BaseMetric*

Quantus Non-Sensitivity metric.

This metric evaluates whether near-zero attribution values are assigned to features on which the model output is not functionally dependent. For each observation, Quantus compares features whose attribution is below a threshold with features whose perturbation produces only a negligible change in the selected model output.

The score measures disagreement between both sets of features. Lower values indicate better agreement between near-zero attributions and features that do not significantly affect the model output.

The metric is based on the Non-Sensitivity metric discussed by Nguyen and Rodríguez Martínez (2020), Ancona et al. (2019), and Montavon et al. (2018), as implemented in Quantus.

Parameters

- **context** (MetricContext) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
  - `eps` : float, optional Threshold used to decide whether an attribution value is considered negligible and whether a prediction change is considered insignificant. The default value is `1e-5`.
  - `features_in_step` : int, optional Number of features perturbed at each step. The default value is `1`.
  - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `True`.
  - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.
  - `perturb_baseline` : str, optional Baseline value used when perturbing features. Supported values depend on the Quantus perturbation function. Common values are `"black"`, `"white"`, `"mean"`, `"random"` and `"uniform"`. The default value is `"black"`.

  If `None`, an empty dictionary is used.

Notes

The wrapper uses the default functions provided by Quantus. The default perturbation function replaces selected features by the configured baseline, and the default aggregation function in Quantus is `numpy.mean`.

Quantus uses `eps` both to identify near-zero attributions and to decide whether the model output change after perturbation is negligible.

NAME: str = 'NonSensitivity'

run()

Compute the Non-Sensitivity metric.

The method selects the observations defined in the metric context and passes their input data, labels and attribution values to `quantus.NonSensitivity`. Quantus perturbs groups of features, measures the resulting change in the selected model output and compares perturbation-insensitive features with features whose attribution values are below `eps`.

If all attribution values are negative, their treatment depends on the `abs` parameter. Their absolute values are used when `abs=True`; otherwise, the metric is skipped.

The model is set to evaluation mode before the metric is computed.

Returns

Non-Sensitivity score for each evaluated observation. Lower values indicate better agreement between near-zero attributions and features that do not significantly affect the model output.

Return type
List[float]

Raises
MetricSkipped – If all attribution values are negative and `abs` is `False`.

Module contents

4.1.3.2 Module contents

4.1.4 xai_metrics.metrics.robustness package

4.1.4.1 Submodules

xai_metrics.metrics.robustness.local_lipschitz_estimate module

class xai_metrics.metrics.robustness.local_lipschitz_estimate.**LocalLipschitzEstimate**(*context*, *explain_func*, *params=None*)

Bases: *BaseMetric*

Quantus Local Lipschitz Estimate metric.

This metric evaluates the local stability of an explanation by comparing changes in the explanation with changes in the corresponding input. For each observation, Quantus generates neighbouring inputs using Gaussian noise, recomputes their explanations and calculates the ratio between the explanation distance and the input distance. The score is the maximum ratio obtained across the sampled neighbours.

Lower scores indicate that nearby inputs receive similar explanations and therefore correspond to more locally robust explanations.

The metric is based on the Local Lipschitz Estimate proposed by Alvarez-Melis and Jaakkola (2018) and implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations, attribution values and optional device information.
- **explain_func** (*ExplainFunc*) – Function used to generate explanations for perturbed inputs. The function must accept the model, a batch of inputs and optionally a batch of labels or targets, and must return a NumPy array containing the generated attributions.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
  - `nr_samples` : int, optional Number of perturbed samples generated for each observation. The default value is `200`.
  - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
  - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.
  - `perturb_mean` : float, optional Mean of the Gaussian noise used for perturbations. The default value is `0.0`.
  - `perturb_std` : float, optional Standard deviation of the Gaussian noise used for perturbations. The default value is `0.1`.

If None, an empty dictionary is used.

Notes

The wrapper uses the default functions provided by Quantus: the Lipschitz constant as the comparison function, Euclidean distance for the numerator and denominator, the default attribution normalisation function and Gaussian-noise perturbations.

The score is a finite-sample estimate based on the generated neighbours, rather than the exact maximum over the complete local neighbourhood.

Raises
ValueError – If explain_func is not provided.

Parameters

- **context** (MetricContext)
- **explain_func** (ExplainFunc)
- **params** (*Mapping[str, Any] | None*)

NAME: str = 'LocalLipschitzEstimate'

run()

Compute the Local Lipschitz Estimate metric.

The method passes the selected inputs, target labels, original attributions and explanation function to quantus.LocalLipschitzEstimate. Quantus samples neighbouring inputs, recomputes their explanations and returns the maximum ratio between explanation and input changes for each observation.

If all attribution values are negative, their absolute values are used when abs=True; otherwise, the metric is skipped. The model is set to training mode and the device stored in the context is forwarded to Quantus.

Returns
Local Lipschitz Estimate score for each evaluated observation. Lower values indicate greater local explanation stability.

Return type
List[float]

Raises
MetricSkipped – If all attribution values are negative and abs is False.

xai_metrics.metrics.robustness.max_sensitivity module

class xai_metrics.metrics.robustness.max_sensitivity.**MaxSensitivity**(*context*, *explain_func*, *params=None*)

Bases: *BaseMetric*

Quantus Max-Sensitivity metric.

This metric evaluates explanation robustness by measuring the largest relative change in an explanation under small input perturbations. For each observation, Quantus generates several perturbed inputs, recomputes their explanations and compares them with the original explanation. The score is the maximum ratio between the norm of the explanation change and the norm of the original explanation.

Lower scores indicate more robust explanations, while higher scores indicate greater sensitivity to input perturbations.

The metric is based on Max-Sensitivity proposed by Yeh et al. (2019) and discussed by Bhatt et al. (2020), as implemented in Quantus.

Parameters

- **context** (MetricContext) – Shared metric evaluation context. It must contain the model, X_test, y_test, selected observations, attribution values and optional device information.
- **explain_func** (ExplainFunc) – Function used to generate explanations for perturbed inputs. The function must be compatible with Quantus explanation functions and return a NumPy array containing the generated attributions.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
  - nr_samples : int, optional Number of perturbed samples generated for each observation. The default value is 200.
  - abs : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is False.
  - normalise : bool, optional Whether to normalise the attribution values before computing the metric. The default value is False.
  - lower_bound : float, optional Lower bound of the uniform noise used for perturbations. The default value is 0.2.
  - upper_bound : float or None, optional Upper bound of the uniform noise used for perturbations. If None, Quantus uses its default behaviour. The default value is None.

  If None, an empty dictionary is used.

Notes

The wrapper uses the default functions provided by Quantus: element-wise difference to compare explanations, the Frobenius norm for the numerator and denominator, the default normalisation function and uniform-noise perturbations.

Raises

ValueError – If explain_func is not provided.

Parameters

- **context** (MetricContext)
- **explain_func** (ExplainFunc)
- **params** (*Mapping[str, Any] | None*)

NAME: str = 'MaxSensitivity'

run()

Compute the Max-Sensitivity metric.

The method passes the selected inputs, target labels, original attributions and explanation function to quantus.MaxSensitivity. Quantus repeatedly perturbs each input, recomputes its explanation and returns the maximum relative explanation change across the configured perturbations.

If all attribution values are negative, their treatment depends on the abs parameter. Their absolute values are used when abs=True; otherwise, the metric is skipped.

The model is set to training mode before the metric is computed. The device stored in the metric context is forwarded to Quantus.

Returns

Max-Sensitivity score for each evaluated observation. Lower values indicate explanations that are more robust to small random input perturbations.

Return type
List[float]

Raises
MetricSkipped – If all attribution values are negative and `abs` is `False`.

xai_metrics.metrics.robustness.relative_input_stability module

class xai_metrics.metrics.robustness.relative_input_stability.**RelativeInputStability**(*context*, *explain_func*, *params=None*)

Bases: *BaseMetric*

Quantus Relative Input Stability metric.

This metric evaluates explanation robustness by comparing the relative change in an explanation with the relative change in its input. For each observation, Quantus generates several perturbed inputs, recomputes their explanations and returns the maximum ratio between both relative changes.

Lower scores indicate more stable explanations, while higher scores indicate greater sensitivity to input perturbations.

The metric is based on Relative Input Stability proposed by Agarwal et al. (2022), as implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations, attribution values and optional device information.
- **explain_func** (*ExplainFunc*) – Function used to generate explanations for perturbed inputs. The function must be compatible with Quantus explanation functions and return a NumPy array containing the generated attributions.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
  - `nr_samples` : int, optional Number of perturbed samples generated for each observation. The default value is `200`.
  - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
  - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `False`.

  If `None`, an empty dictionary is used.

Notes

Quantus uses uniform-noise perturbations with an upper bound of `0.2`. When normalisation is enabled, the default function is `normalise_by_average_second_moment_estimate`.

A constant of `1e-6` is used to avoid division by zero. By default, perturbations that change the model prediction produce `nan` scores.

Raises
ValueError – If `explain_func` is not provided.

Parameters

- **context** (*MetricContext*)

- **explain_func** (ExplainFunc)
- **params** (*Mapping[str, Any] | None*)

NAME: str = 'RelativeInputStability'

run()

Compute the Relative Input Stability metric.

The method passes the selected inputs, target labels, original attributions and explanation function to `quantus.RelativeInputStability`. Quantus perturbs each input, recomputes its explanation and returns the maximum ratio between the relative explanation change and the relative input change.

If all attribution values are negative, their treatment depends on the `abs` parameter. Their absolute values are used when `abs=True`; otherwise, the metric is skipped.

The model is set to evaluation mode before the metric is computed. The device stored in the metric context is forwarded to Quantus.

Returns
Relative Input Stability score for each evaluated observation. Lower values indicate more stable explanations. A score may be `nan` when a perturbation changes the model prediction.

Return type
list[float]

Raises
MetricSkipped – If all attribution values are negative and `abs` is `False`.

xai_metrics.metrics.robustness.relative_output_stability module

class xai_metrics.metrics.robustness.relative_output_stability.**RelativeOutputStability**(*context*, *explain_func*, *params=None*)

Bases: *BaseMetric*

Quantus Relative Output Stability metric.

This metric evaluates explanation robustness by comparing the relative change in an explanation with the corresponding change in the model output logits. For each observation, Quantus perturbs the input, recomputes its explanation and returns the maximum ratio obtained across the sampled perturbations.

Lower scores indicate that explanations change less relative to changes in the model output and are therefore considered more stable.

The metric is based on Relative Output Stability proposed by Agarwal et al. (2022), as implemented in Quantus.

Parameters

- **context** (MetricContext) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations, attribution values and optional device information.
- **explain_func** (ExplainFunc) – Function used to generate explanations for perturbed inputs. The function must be compatible with Quantus explanation functions and return a NumPy array containing the generated attributions.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
  - `nr_samples` : int, optional Number of perturbed samples generated for each observation. The default value is `200`.

- abs : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is False.
- normalise : bool, optional Whether to normalise the attribution values before computing the metric. The default value is False.

If None, an empty dictionary is used.

Notes

Quantus uses uniform-noise perturbations with an upper bound of 0.2. When normalisation is enabled, the default function is normalise_by_average_second_moment_estimate.

A constant of 1e-6 is used to avoid division by zero. By default, perturbations that change the predicted class produce nan scores.

Raises
ValueError – If explain_func is not provided.

Parameters

- **context** (MetricContext)
- **explain_func** (ExplainFunc)
- **params** (*Mapping[str, Any] | None*)

NAME: str = 'RelativeOutputStability'

run()

Compute the Relative Output Stability metric.

The method passes the selected inputs, target labels, original attributions and explanation function to quantus.RelativeOutputStability. Quantus repeatedly perturbs each input and returns the maximum ratio between the relative explanation change and the distance between the original and perturbed output-logit vectors.

If all attribution values are negative, their treatment depends on the abs parameter. Their absolute values are used when abs=True; otherwise, the metric is skipped.

The model is set to evaluation mode before the metric is computed. The device stored in the metric context is forwarded to Quantus.

Returns
Relative Output Stability score for each evaluated observation. Lower values indicate more stable explanations. A score may be nan when a perturbation changes the predicted class.

Return type
list[float]

Raises
MetricSkipped – If all attribution values are negative and abs is False.

4.1.4.2 Module contents

4.1.5 xai_metrics.metrics.sensitivity package

4.1.5.1 Submodules

xai_metrics.metrics.sensitivity.avg_sensitivity module

class xai_metrics.metrics.sensitivity.avg_sensitivity.**AvgSensitivity**(*context*, *explain_func*, *params=None*)

Bases: *BaseMetric*

Quantus Average Sensitivity metric.

This metric evaluates explanation robustness by measuring the average relative change in an explanation when small random perturbations are applied to its input. For each observation, Quantus generates several perturbed inputs, recomputes their explanations using `explain_func` and compares them with the original explanation.

The sensitivity of each perturbation is computed from the norm of the explanation difference relative to the norm of the original explanation. The final score is the average over all sampled perturbations.

Lower scores indicate more robust explanations, whereas higher scores indicate greater sensitivity to small input changes.

The metric is based on Average Sensitivity proposed by Yeh et al. (2019) and discussed by Bhatt et al. (2020), as implemented in Quantus.

Initialize the Average Sensitivity metric.

Parameters

- **context** (MetricContext) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations, attribution values and optional device information.
- **explain_func** (ExplainFunc) – Function used to generate explanations for perturbed inputs. The function must be compatible with Quantus explanation functions and return a NumPy array containing the generated attributions.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
  - `nr_samples` : int, optional Number of perturbed samples generated for each observation. The default value is `200`.
  - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
  - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `False`.
  - `lower_bound` : float, optional Lower bound of the uniform noise used for perturbations. The default value is `0.2`.
  - `upper_bound` : float or None, optional Upper bound of the uniform noise used for perturbations. If `None`, Quantus uses its default behaviour. The default value is `None`.

  If `None`, an empty dictionary is used.

Notes

The wrapper uses the default functions provided by Quantus: element-wise difference for comparing explanations, the Frobenius norm for the numerator and denominator, the default normalisation function and uniform-noise perturbations.

Raises

ValueError – If `explain_func` is not provided.

Parameters

- **context** (MetricContext)
- **explain_func** (ExplainFunc)

- **params** (*Mapping[str, Any] | None*)

NAME: str = 'AvgSensitivity'

run()

Compute the Average Sensitivity metric.

The method passes the selected inputs, target labels, original attributions and explanation function to `quantus.AvgSensitivity`. Quantus repeatedly perturbs each input, recomputes its explanation and averages the relative explanation changes across the configured number of samples.

If all attribution values are negative, their absolute values are used when `abs=True`; otherwise, the metric is skipped. The model is set to training mode and the device stored in the context is forwarded to Quantus.

Returns

Average Sensitivity score for each evaluated observation. Lower values indicate greater robustness to random input perturbations.

Return type

List[float]

Raises

MetricSkipped – If all attribution values are negative and `abs` is `False`.

4.1.5.2 Module contents

4.2 Submodules

4.2.1 xai_metrics.metrics.autodiscover module

xai_metrics.metrics.autodiscover.**autodiscover_metrics**(*package*)

Import all submodules of a metrics package.

This function recursively walks through `package` and imports every discovered module. Importing the modules triggers any registration logic executed at import time, such as `@register_metric`.

Parameters

package (*ModuleType*) – Imported package whose submodules should be discovered. The package must expose a `__path__` attribute.

Raises

- **AttributeError** – If `package` is not a package with a `__path__` attribute.
- **ImportError** – If a discovered submodule cannot be imported.

4.3 Module contents

CHAPTER

FIVE

XAI_METRICS.REPORTING PACKAGE

5.1 Submodules

5.1.1 xai_metrics.reporting.reporting module

xai_metrics.reporting.reporting.**build_observation_reports**(*context_outputs*)

Build observation-level metric reports from context outputs.

Observation-level reports preserve one metric value per evaluated observation instead of aggregating metric outputs. Reports are grouped by metric, dataset and model. Each dataframe contains one row per observation and one column per XAI method.

Metric outputs that cannot be aligned with the observation list are stored as `None` for the corresponding XAI method. Different parameter configurations for the same metric are kept as separate rows because the grouping includes the serialized `metric_params` signature.

Parameters

context_outputs (*Sequence[Mapping[str, Any]]*) – Sequence of context output mappings. Each element must contain:

- `metadata` : Mapping[str, Any] Metadata identifying the dataset, model and XAI method.
- `results` : Sequence[Mapping[str, Any]] Metric result entries. Each entry must contain `metric` and `value` and may contain `metric_params`.
- `observations` : Sequence[Any] Observation identifiers corresponding to the metric outputs.

Returns

Nested report dictionary. The first level is indexed by metric name, the second by dataset name and the third by model name.

Return type

Dict[str, Dict[str, Dict[str, pandas.DataFrame]]]

Raises

- **ValueError** – If a context output does not include `observations`, if observation indexes differ across XAI methods for the same report, or if a duplicated observation-level result is found for the same metric, parameter configuration and XAI method.
- **KeyError** – If required metadata or metric result fields are missing.

xai_metrics.reporting.reporting.**build_reports**(*context_outputs*)

Build summary metric reports from context outputs.

Each context output must contain a metadata mapping and a results sequence. Metadata must define dataset_name, model_name and xai_method_name. Each metric result in results must contain the fields metric and value and may contain metric_params.

Results are grouped by metric, dataset, model and metric parameters. Each generated dataframe contains the fixed columns dataset_name, model_name, metric and metric_params, followed by one result column for each XAI method.

Numeric metric values are flattened, nan values are removed and the remaining values are averaged. Non-numeric values are serialized without aggregation. If a metric value is a mapping, each entry is expanded into a separate report row named "<metric_name>.<result_key>".

Parameters

context_outputs (*Sequence[Mapping[str, Any]]*) – Sequence of context output mappings. Each element must contain:

- metadata : Mapping[str, Any] Metadata identifying the dataset, model and XAI method.
- results : Sequence[Mapping[str, Any]] Metric result entries. Each entry must contain metric and value and may contain metric_params.

Returns

Nested report dictionary. The first level is indexed by dataset name and the second level by model name. Each dataframe has one row per report metric and parameter configuration, and one result column per XAI method.

Return type

Dict[str, Dict[str, pandas.DataFrame]]

Raises

- **KeyError** – If a context output or metric result does not contain the required fields.
- **ValueError** – If an XAI method name conflicts with a reserved report column or if a duplicated result is found for the same metric, parameter configuration and XAI method.

xai_metrics.reporting.reporting.**save_attributions**(*context_outputs*, *output_dir*)

Save generated attributions to CSV files.

For each context output, the attribution matrix is converted into a dataframe using the stored observation identifiers as index and feature names as columns. Files are saved under <output_dir>/<dataset>/<model>/<xai_method>/.

Parameters

- **context_outputs** (*Sequence[Mapping[str, Any]]*) – Sequence of context output mappings. Each element must contain metadata, attributions, observations and features.
- **output_dir** (*str or pathlib.Path*) – Root directory where attribution files will be saved. Missing directories are created automatically.

Returns

Dictionary mapping each XAI method name to the path of the generated attribution CSV file.

Return type

Dict[str, str]

Raises

KeyError – If any required field is missing from a context output.

xai_metrics.reporting.reporting.**save_reports**(*reports*, *output_dir*, *report_name='xai_metrics_report'*, *observation_reports=None*, *experiment_started_at=None*)

Save summary reports and optional observation-level reports.

Summary report dataframes for every dataset and model are concatenated into a single report. Rows are sorted by dataset name, model name and metric name, while XAI method columns are sorted alphabetically.

The summary report is saved as both CSV and JSON. Structured CSV cell values, such as metric parameter dictionaries, are encoded as JSON strings. The JSON file contains the complete summary report as serialized records under the `"report"` key.

If `observation_reports` is provided, one CSV file is saved for each metric. These files preserve observation-level metric values and are stored in metric-specific subdirectories.

Parameters

- **reports** (*Mapping[str, Mapping[str, pandas.DataFrame]]*) – Nested dictionary of summary report dataframes, usually returned by *build_reports()*. The first level must be indexed by dataset name and the second level by model name.
- **output_dir** (*str or pathlib.Path*) – Directory where the report files will be saved. The directory and its missing parent directories are created automatically.
- **report_name** (*str, default="xai_metrics_report"*) – Base filename used for the generated summary report files. A timestamp suffix is added automatically.
- **observation_reports** (*Mapping[str, Mapping[str, Mapping[str, pandas.DataFrame]]] or None, optional*) – Optional nested dictionary of observation-level reports, usually returned by *build_observation_reports()*.
- **experiment_started_at** (*Any or None, optional*) – Timestamp used to generate deterministic report filenames. If `None`, the current datetime is used.

Returns

Dictionary containing the generated summary report paths under `"csv"` and `"json"`. If observation reports are saved, the dictionary also contains `"observation_csv"`, mapping each metric name to its observation-level CSV path.

Return type

Dict[str, str]

Raises

ValueError – If `reports` does not contain any report dataframes to save.

5.2 Module contents

CHAPTER

SIX

XAI_METRICS.RUNNER PACKAGE

6.1 Submodules

6.1.1 xai_metrics.runner.runner module

xai_metrics.runner.runner.**run_evaluation**(*context=None*, *metadata=None*, *selected_metrics=None*, *config='config.yaml'*, *report_output_dir=PosixPath('results/reports')*, ***runtime_kwargs*)

Run the XAI metric evaluation pipeline.

The function discovers all metric modules, loads the experiment configuration, builds or accepts one or more metric contexts, resolves the metrics that can be executed, injects runtime dependencies and evaluates each selected metric.

When `context` is `None`, metric contexts are built from `config` through `ConfigController`. When a context is supplied directly, `metadata` must also be provided.

Runtime keyword arguments are forwarded as dependencies to metric constructors when their names match constructor parameters. The `model_loader` argument is passed to the configuration controller.

The special `explain_funcs` argument may contain either a global mapping from XAI method names to explanation functions or a nested mapping from dataset names to XAI method names. Names are matched case-insensitively. The function corresponding to the current context is injected into compatible metrics under the `explain_func` dependency.

Metric results are stored as a list of dictionaries. Each result entry contains the metric name, the metric parameters used in that execution and the returned value. Metrics that raise `MetricSkipped` are recorded in `skipped` with their parameters and skip reason, without stopping the remaining metrics.

Summary reports and observation-level reports are built from the collected outputs. If `report_output_dir` is not `None`, reports are saved to disk after metric results become available.

Parameters

- **context** (*MetricContext or None, optional*) – Pre-built metric evaluation context. If `None`, one or more contexts are constructed from `config`.
- **metadata** (*Mapping[str, Any] or None, optional*) – Metadata associated with a directly supplied `context`. It must contain `dataset_name`, `model_name` and `xai_method_name`. Ignored when contexts are built from the configuration.
- **selected_metrics** (*Iterable[str] or None, optional*) – Metric names to execute. Only metrics that are also configured and registered are evaluated. If `None`, all configured and registered metrics are evaluated.

- **config** (*Mapping[str, Any], str or pathlib.Path, default="config.yaml"*) – Experiment configuration source. It may be a mapping or the path to a YAML configuration file.
- **report_output_dir** (*str, pathlib.Path or None, optional*) – Directory where the generated reports are saved. If `None`, reports are built but not written to disk. The default directory is `results/reports`.
- ****runtime_kwargs** (*Any*) – Runtime dependencies made available to metric constructors. Supported values mostly depend on the selected metrics.

  Two arguments receive special handling:

  - `model_loader` : Callable, optional Function used by the configuration controller to load models.
  - `explain_funcs` : Mapping[str, Callable] or Mapping[str, Mapping[str, Callable]], optional Mapping from XAI method names to explanation functions, or nested mapping from dataset names to XAI method names.

Returns

Dictionary containing the following entries:

- `"contexts"` : List[Dict[str, Any]] Raw output for each evaluated context. Every context output contains `metadata`, `observations`, `results` and `skipped`. `results` is a list of dictionaries with `metric`, `metric_params` and `value`. `skipped` is a list of dictionaries with `metric`, `metric_params` and `reason`.
- `"reports"` : Dict[str, Dict[str, pandas.DataFrame]] Summary report dataframes grouped first by dataset name and then by model name.
- `"observation_reports"` : Dict[str, Dict[str, Dict[str, pandas.DataFrame]]] Observation-level reports grouped by metric, dataset and model.
- `"report_paths"` : Dict[str, Any] Paths of the saved report files. The dictionary is empty when `report_output_dir` is `None`.

Return type
Dict[str, Any]

Raises

- **ValueError** – If metadata required for a directly supplied context is missing, if no explanation function is available for the current XAI method, or if the configuration or report data are invalid.
- **TypeError** – If context construction loads an object that is not a supported model type.
- **RuntimeError** – If context construction requests a device that is not available.

Warns
UserWarning – If configured or selected metrics cannot be executed because they are missing from the configuration or metric registry.

xai_metrics.runner.runner.**run_explanation**(*context=None*, *selected_explainers=None*, *config='config.yaml'*, *attribution_output_dir=PosixPath('results/attributions')*, ***runtime_kwargs*)

Run the XAI explanation generation pipeline.

The function discovers all explainer modules, loads the experiment configuration, builds or accepts one or more explainer contexts, resolves the explainers that can be executed and generates attribution values for each selected explainer.

When `context` is `None`, explainer contexts are built from `config` through `ConfigController`. When a context is supplied directly, the configuration must provide `dataset_name` and `model_name` in its `context` section.

Generated attributions are stored together with the parameters configured for each explainer. Explainers that raise `ExplainerSkipped` are recorded separately and do not stop the remaining explainers.

If `attribution_output_dir` is not `None`, generated attribution matrices are also saved as CSV files.

Parameters

- **context** (*ExplainerContext or None, optional*) – Pre-built explainer context. If `None`, one or more contexts are constructed from `config`.
- **selected_explainers** (*Iterable[str] or None, optional*) – Explainer names to execute. Only explainers that are also configured and registered are run. If `None`, all configured and registered explainers are executed.
- **config** (*Mapping[str, Any], str or pathlib.Path, default="config.yaml"*) – Experiment configuration source. It may be a mapping or the path to a YAML configuration file.
- **attribution_output_dir** (*str, pathlib.Path or None, optional*) – Directory where generated attribution CSV files are saved. If `None`, attributions are returned but not written to disk. The default directory is `results/attributions`.
- ****runtime_kwargs** (*Any*) – Runtime objects used by the explanation workflow. The `model_loader` argument receives special handling and is passed to `ConfigController`.

Returns

Dictionary containing the following entries:

- `"contexts"` : List[Dict[str, Any]] Raw output for each explainer context. Each entry contains `metadata`, `attributions`, `explainer_params` and `skipped`.
- `"attribution_paths"` : Dict[str, str] Paths of the saved attribution files. The dictionary is empty when `attribution_output_dir` is `None`.

Return type
Dict[str, Any]

Raises

- **ValueError** – If the configuration has no `context` section, if required metadata are missing, or if an `ExplainerContext` does not include a model or `X_batch`.
- **RuntimeError** – If context construction requests a device that is not available.

Warns
UserWarning – If configured or selected explainers cannot be executed because they are missing from the configuration or explainer registry.

6.2 Module contents

PYTHON MODULE INDEX

X

xai_metrics.base, 6
xai_metrics.base.base_explainer, 1
xai_metrics.base.base_metric, 3
xai_metrics.base.explainer_registry, 4
xai_metrics.base.metric_registry, 5
xai_metrics.base.types, 6
xai_metrics.config, 9
xai_metrics.config.config_controller, 7
xai_metrics.explainers, 17
xai_metrics.explainers.autodiscover, 11
xai_metrics.explainers.breakdown, 11
xai_metrics.explainers.lime, 12
xai_metrics.explainers.maple, 14
xai_metrics.explainers.shap, 16
xai_metrics.metrics, 40
xai_metrics.metrics.autodiscover, 40
xai_metrics.metrics.complexity, 21
xai_metrics.metrics.complexity.complexity_metric, 19
xai_metrics.metrics.complexity.sparseness, 20
xai_metrics.metrics.faithfulness, 30
xai_metrics.metrics.faithfulness.consistency, 21
xai_metrics.metrics.faithfulness.faithfulness, 22
xai_metrics.metrics.faithfulness.faithfulness_estimate, 23
xai_metrics.metrics.faithfulness.monotonicity, 24
xai_metrics.metrics.faithfulness.monotonicity_correlation, 25
xai_metrics.metrics.faithfulness.monotonicity_metric, 27
xai_metrics.metrics.faithfulness.sensitivity_n, 28
xai_metrics.metrics.faithfulness.sufficiency, 29
xai_metrics.metrics.fidelity, 33
xai_metrics.metrics.fidelity.completeness, 31
xai_metrics.metrics.fidelity.completeness.completeness_metric, 30
xai_metrics.metrics.fidelity.soundness, 33
xai_metrics.metrics.fidelity.soundness.non_sensitivity, 31
xai_metrics.metrics.robustness, 38
xai_metrics.metrics.robustness.local_lipschitz_estimate, 33
xai_metrics.metrics.robustness.max_sensitivity, 34
xai_metrics.metrics.robustness.relative_input_stability, 36
xai_metrics.metrics.robustness.relative_output_stability, 37
xai_metrics.metrics.sensitivity, 40
xai_metrics.metrics.sensitivity.avg_sensitivity, 38
xai_metrics.reporting, 43
xai_metrics.reporting.reporting, 41
xai_metrics.runner, 47
xai_metrics.runner.runner, 45

INDEX

A

as_explain_func() (*xai_metrics.base.base_explainer.BaseExplainer method*), 1
autodiscover_explainers() (*in module xai_metrics.explainers.autodiscover*), 11
autodiscover_metrics() (*in module xai_metrics.metrics.autodiscover*), 40
AvgSensitivity (*class in xai_metrics.metrics.sensitivity.avg_sensitivity*), 38

B

BaseExplainer (*class in xai_metrics.base.base_explainer*), 1
BaseMetric (*class in xai_metrics.base.base_metric*), 3
BreakDownExplainer (*class in xai_metrics.explainers.breakdown*), 11
build_explainers_context() (*xai_metrics.config.config_controller.ConfigController method*), 7
build_explainers_contexts() (*xai_metrics.config.config_controller.ConfigController method*), 8
build_explainers_from_config() (*in module xai_metrics.base.explainer_registry*), 4
build_metric_context() (*xai_metrics.config.config_controller.ConfigController method*), 8
build_metric_contexts() (*xai_metrics.config.config_controller.ConfigController method*), 8
build_metrics_from_config() (*in module xai_metrics.base.metric_registry*), 5
build_observation_reports() (*in module xai_metrics.reporting.reporting*), 41
build_reports() (*in module xai_metrics.reporting.reporting*), 41

C

Completeness (*class in xai_metrics.metrics.fidelity.completeness.completeness_metric*), 30
Complexity (*class in xai_metrics.metrics.complexity.complexity_metric*), 19
ConfigController (*class in xai_metrics.config.config_controller*), 7
Consistency (*class in xai_metrics.metrics.faithfulness.consistency*), 21

D

default_model_loader() (*in module xai_metrics.config.config_controller*), 9
device (*xai_metrics.base.base_explainer.ExplainerContext attribute*), 2
device (*xai_metrics.base.base_metric.MetricContext attribute*), 3

E

explain() (*xai_metrics.base.base_explainer.BaseExplainer method*), 1
explain() (*xai_metrics.explainers.breakdown.BreakDownExplainer method*), 12
explain() (*xai_metrics.explainers.lime.LIMEExplainer method*), 14
explain() (*xai_metrics.explainers.maple.MAPLEExplainer method*), 15
explain() (*xai_metrics.explainers.shap.SHAPExplainer method*), 16
ExplainerContext (*class in xai_metrics.base.base_explainer*), 2
ExplainerSkipped, 2
ExplainFunc (*in module xai_metrics.base.types*), 6

F

Faithfulness (*class in xai_metrics.metrics.faithfulness.faithfulness*), 22
FaithfulnessEstimate (*class in xai_metrics.metrics.faithfulness.faithfulness_estimate*), 23

G

get_explainers_config()

(*xai_metrics.config.config_controller.ConfigController method*), 9
get_metrics_config() (*xai_metrics.config.config_controller.ConfigController method*), 9

L

LIMEExplainer (*class in xai_metrics.explainers.lime*), 12
list_explainers() (*in module xai_metrics.base.explainer_registry*), 4
list_metrics() (*in module xai_metrics.base.metric_registry*), 5
LocalLipschitzEstimate (*class in xai_metrics.metrics.robustness.local_lipschitz_estimate*), 33

M

MAPLEExplainer (*class in xai_metrics.explainers.maple*), 14
MaxSensitivity (*class in xai_metrics.metrics.robustness.max_sensitivity*), 34
MetricContext (*class in xai_metrics.base.base_metric*), 3
MetricSkipped, 3
model (*xai_metrics.base.base_explainer.ExplainerContext attribute*), 2
module
xai_metrics.base, 6
xai_metrics.base.base_explainer, 1
xai_metrics.base.base_metric, 3
xai_metrics.base.explainer_registry, 4
xai_metrics.base.metric_registry, 5
xai_metrics.base.types, 6
xai_metrics.config, 9
xai_metrics.config.config_controller, 7
xai_metrics.explainers, 17
xai_metrics.explainers.autodiscover, 11
xai_metrics.explainers.breakdown, 11
xai_metrics.explainers.lime, 12
xai_metrics.explainers.maple, 14
xai_metrics.explainers.shap, 16
xai_metrics.metrics, 40
xai_metrics.metrics.autodiscover, 40
xai_metrics.metrics.complexity, 21
xai_metrics.metrics.complexity.complexity_metric, 19
xai_metrics.metrics.complexity.sparseness, 20
xai_metrics.metrics.faithfulness, 30
xai_metrics.metrics.faithfulness.consistency, 21
xai_metrics.metrics.faithfulness.faithfulness, 22
xai_metrics.metrics.faithfulness.faithfulness_estimate 23
xai_metrics.metrics.faithfulness.monotonicity, 24
xai_metrics.metrics.faithfulness.monotonicity_correlat 25
xai_metrics.metrics.faithfulness.monotonicity_metric, 27
xai_metrics.metrics.faithfulness.sensitivity_n, 28
xai_metrics.metrics.faithfulness.sufficiency, 29
xai_metrics.metrics.fidelity, 33
xai_metrics.metrics.fidelity.completeness, 31
xai_metrics.metrics.fidelity.completeness.completeness 30
xai_metrics.metrics.fidelity.soundness, 33
xai_metrics.metrics.fidelity.soundness.non_sensitivity 31
xai_metrics.metrics.robustness, 38
xai_metrics.metrics.robustness.local_lipschitz_estimat 33
xai_metrics.metrics.robustness.max_sensitivity, 34
xai_metrics.metrics.robustness.relative_input_stabilit 36
xai_metrics.metrics.robustness.relative_output_stabili 37
xai_metrics.metrics.sensitivity, 40
xai_metrics.metrics.sensitivity.avg_sensitivity, 38
xai_metrics.reporting, 43
xai_metrics.reporting.reporting, 41
xai_metrics.runner, 47
xai_metrics.runner.runner, 45
Monotonicity (*class in xai_metrics.metrics.faithfulness.monotonicity*), 24
MonotonicityCorrelation (*class in xai_metrics.metrics.faithfulness.monotonicity_correlation*), 25
MonotonicityMetric (*class in xai_metrics.metrics.faithfulness.monotonicity_metric*), 27

N

NAME (*xai_metrics.base.base_explainer.BaseExplainer attribute*), 1
NAME (*xai_metrics.base.base_metric.BaseMetric attribute*), 3

NAME (*xai_metrics.explainers.breakdown.BreakDownExplainer attribute*), 12
NAME (*xai_metrics.explainers.lime.LIMEExplainer attribute*), 14
NAME (*xai_metrics.explainers.maple.MAPLEExplainer attribute*), 15
NAME (*xai_metrics.explainers.shap.SHAPExplainer attribute*), 16
NAME (*xai_metrics.metrics.complexity.complexity_metric.Complexity attribute*), 19
NAME (*xai_metrics.metrics.complexity.sparseness.Sparseness attribute*), 20
NAME (*xai_metrics.metrics.faithfulness.consistency.Consistency attribute*), 21
NAME (*xai_metrics.metrics.faithfulness.faithfulness.Faithfulness attribute*), 22
NAME (*xai_metrics.metrics.faithfulness.faithfulness_estimate.FaithfulnessEstimate attribute*), 24
NAME (*xai_metrics.metrics.faithfulness.monotonicity.Monotonicity attribute*), 25
NAME (*xai_metrics.metrics.faithfulness.monotonicity_correlation.MonotonicityCorrelation attribute*), 26
NAME (*xai_metrics.metrics.faithfulness.monotonicity_metric.MonotonicityMetric attribute*), 27
NAME (*xai_metrics.metrics.faithfulness.sensitivity_n.SensitivityN attribute*), 28
NAME (*xai_metrics.metrics.faithfulness.sufficiency.Sufficiency attribute*), 29
NAME (*xai_metrics.metrics.fidelity.completeness.completeness_metric.Completeness attribute*), 31
NAME (*xai_metrics.metrics.fidelity.soundness.non_sensitivity.NonSensitivity attribute*), 32
NAME (*xai_metrics.metrics.robustness.local_lipschitz_estimate.LocalLipschitzEstimate attribute*), 34
NAME (*xai_metrics.metrics.robustness.max_sensitivity.MaxSensitivity attribute*), 35
NAME (*xai_metrics.metrics.robustness.relative_input_stability.RelativeInputStability attribute*), 37
NAME (*xai_metrics.metrics.robustness.relative_output_stability.RelativeOutputStability attribute*), 38
NAME (*xai_metrics.metrics.sensitivity.avg_sensitivity.AvgSensitivity attribute*), 40
NonSensitivity (*class in xai_metrics.metrics.fidelity.soundness.non_sensitivity*), 31

R

register_explainer() (*in module xai_metrics.base.explainer_registry*), 4
register_metric() (*in module xai_metrics.base.metric_registry*), 5
RelativeInputStability (*class in xai_metrics.metrics.robustness.relative_input_stability*), 36
RelativeOutputStability (*class in xai_metrics.metrics.robustness.relative_output_stability*), 37
run() (*xai_metrics.base.base_metric.BaseMetric method*), 3
run() (*xai_metrics.metrics.complexity.complexity_metric.Complexity method*), 19
run() (*xai_metrics.metrics.complexity.sparseness.Sparseness method*), 20
run() (*xai_metrics.metrics.faithfulness.consistency.Consistency method*), 21
run() (*xai_metrics.metrics.faithfulness.faithfulness.Faithfulness method*), 22
run() (*xai_metrics.metrics.faithfulness.faithfulness_estimate.FaithfulnessEstimate method*), 24
run() (*xai_metrics.metrics.faithfulness.monotonicity.Monotonicity method*), 25
run() (*xai_metrics.metrics.faithfulness.monotonicity_correlation.MonotonicityCorrelation method*), 26
run() (*xai_metrics.metrics.faithfulness.monotonicity_metric.MonotonicityMetric method*), 27
run() (*xai_metrics.metrics.faithfulness.sensitivity_n.SensitivityN method*), 28
run() (*xai_metrics.metrics.faithfulness.sufficiency.Sufficiency method*), 29
run() (*xai_metrics.metrics.fidelity.completeness.completeness_metric.Completeness method*), 31
run() (*xai_metrics.metrics.fidelity.soundness.non_sensitivity.NonSensitivity method*), 32
run() (*xai_metrics.metrics.robustness.local_lipschitz_estimate.LocalLipschitzEstimate method*), 34
run() (*xai_metrics.metrics.robustness.max_sensitivity.MaxSensitivity method*), 35
run() (*xai_metrics.metrics.robustness.relative_input_stability.RelativeInputStability method*), 37
run() (*xai_metrics.metrics.robustness.relative_output_stability.RelativeOutputStability method*), 38
run() (*xai_metrics.metrics.sensitivity.avg_sensitivity.AvgSensitivity method*), 40
run_evaluation() (*in module xai_metrics.runner.runner*), 45
run_explanation() (*in module xai_metrics.runner.runner*), 46

S

save_attributions() (*in module xai_metrics.reporting.reporting*), 42
save_reports() (*in module xai_metrics.reporting.reporting*), 42
SensitivityN (*class in xai_metrics.metrics.faithfulness.sensitivity_n*), 28
SHAPExplainer (*class in xai_metrics.explainers.shap*), 16

Sparseness (*class in xai_metrics.metrics.complexity.sparseness*), 20
Sufficiency (*class in xai_metrics.metrics.faithfulness.sufficiency*), 29

X

X_batch (*xai_metrics.base.base_explainer.ExplainerContext attribute*), 2
xai_metrics.base
    module, 6
xai_metrics.base.base_explainer
    module, 1
xai_metrics.base.base_metric
    module, 3
xai_metrics.base.explainer_registry
    module, 4
xai_metrics.base.metric_registry
    module, 5
xai_metrics.base.types
    module, 6
xai_metrics.config
    module, 9
xai_metrics.config.config_controller
    module, 7
xai_metrics.explainers
    module, 17
xai_metrics.explainers.autodiscover
    module, 11
xai_metrics.explainers.breakdown
    module, 11
xai_metrics.explainers.lime
    module, 12
xai_metrics.explainers.maple
    module, 14
xai_metrics.explainers.shap
    module, 16
xai_metrics.metrics
    module, 40
xai_metrics.metrics.autodiscover
    module, 40
xai_metrics.metrics.complexity
    module, 21
xai_metrics.metrics.complexity.complexity_metric
    module, 19
xai_metrics.metrics.complexity.sparseness
    module, 20
xai_metrics.metrics.faithfulness
    module, 30
xai_metrics.metrics.faithfulness.consistency
    module, 21
xai_metrics.metrics.faithfulness.faithfulness
    module, 22
xai_metrics.metrics.faithfulness.faithfulness_estimate
    module, 23
xai_metrics.metrics.faithfulness.monotonicity
    module, 24
xai_metrics.metrics.faithfulness.monotonicity_correlation
    module, 25
xai_metrics.metrics.faithfulness.monotonicity_metric
    module, 27
xai_metrics.metrics.faithfulness.sensitivity_n
    module, 28
xai_metrics.metrics.faithfulness.sufficiency
    module, 29
xai_metrics.metrics.fidelity
    module, 33
xai_metrics.metrics.fidelity.completeness
    module, 31
xai_metrics.metrics.fidelity.completeness.completeness_met
    module, 30
xai_metrics.metrics.fidelity.soundness
    module, 33
xai_metrics.metrics.fidelity.soundness.non_sensitivity
    module, 31
xai_metrics.metrics.robustness
    module, 38
xai_metrics.metrics.robustness.local_lipschitz_estimate
    module, 33
xai_metrics.metrics.robustness.max_sensitivity
    module, 34
xai_metrics.metrics.robustness.relative_input_stability
    module, 36
xai_metrics.metrics.robustness.relative_output_stability
    module, 37
xai_metrics.metrics.sensitivity
    module, 40
xai_metrics.metrics.sensitivity.avg_sensitivity
    module, 38
xai_metrics.reporting
    module, 43
xai_metrics.reporting.reporting
    module, 41
xai_metrics.runner
    module, 47
xai_metrics.runner.runner
    module, 45

Y

y_background (*xai_metrics.base.base_explainer.ExplainerContext attribute*), 2
y_batch (*xai_metrics.base.base_explainer.ExplainerContext attribute*), 2